

Module 6 Structured Note: Comparing Strings and Checking Palindromes

1. What this module is about

Module 6 teaches two important string operations in C/C++:

1. Comparing two strings
2. Checking whether a string is a palindrome

The main skill is still the same skill from earlier modules:

Start from an index.
Check characters one by one.
Stop when you reach `'\0'`.

But Module 6 adds two new ideas:

Compare two strings from left to right.
Compare one string from both ends to check palindrome.

2. Required ideas from previous modules

Before Module 6, remember these points:

A C-style string is a character array ending with `\0`.

Example:

```
char A[] = "madam";
```

Memory view:

Index:	0	1	2	3	4	5
Value:	m	a	d	a	m	\0

The visible word is:

madam

The `\0` is not part of the word. It only marks the end of the string.

So most string loops look like this:

```
for (i = 0; A[i] != '\0'; i++) {  
    // process A[i]  
}
```

This means:

Start at index 0.
Keep going **while** the current character is not `'\0'`.
Stop when the string ends.

Part A: Comparing Two Strings

3. What does string comparison mean?

String comparison means checking how two strings relate to each other.

Possible results:

Both strings are equal.
First string is smaller.
First string is greater.

In C-style strings, we compare strings character by character.

Example:

```
char A[] = "Painter";  
char B[] = "Painting";
```

Memory idea:

```
A: P a i n t e r \0
B: P a i n t i n g \0
```

We compare:

```
A[0] with B[0]
A[1] with B[1]
A[2] with B[2]
...
```

We stop when:

The characters are different.

or

One or both strings reach '\0'.

4. Why strings are compared character by character

A string is not one simple number.

A string is a group of characters stored in an array.

So this:

```
"Painter"
```

is really:

```
P a i n t e r \0
```

Because of that, we cannot understand the whole string by checking only one character.

For example:

```
Painter
Painting
```

Both strings start with:

```
P a i n t
```

But they are still different because later characters do not match.

So we must compare from left to right until we find a difference or reach the end.

5. When are two strings equal?

Two strings are equal if:

```
Each matching position has the same character.
```

and

```
Both strings reach '\0' at the same time.
```

Example:

```
A = "code"
B = "code"
```

Memory idea:

```
A: c o d e \0
B: c o d e \0
```

Comparison:

```
c == c
o == o
d == d
```

```
e == e  
0 == 0
```

Both strings end at the same time, so they are equal.

6. When are two strings not equal?

Two strings are not equal if:

A mismatch is found.

Example:

```
A = "Painter"  
B = "Painting"
```

Comparison:

```
P == P  
a == a  
i == i  
n == n  
t == t  
e != i
```

At index **5**, the characters are different:

```
A[5] = 'e'  
B[5] = 'i'
```

So the strings are not equal.

7. ASCII order in string comparison

When two characters are different, C-style comparison can use ASCII values to decide which string is smaller or greater.

Example:

```
'e' = 101  
'i' = 105
```

Since:

```
101 < 105
```

we say:

```
'e' is smaller than 'i'
```

So:

```
"Painter" is smaller than "Painting"
```

because the first mismatch is:

```
e < i
```

Important idea:

```
The first different character decides the result.
```

Characters after that do not matter.

8. Code: Comparing two strings

```
#include <stdio.h>  
  
int main() {  
    char A[] = "Painter";  
    char B[] = "Painting";  
    int i, j;  
  
    for (i = 0, j = 0; A[i] != '\0' && B[j] != '\0'; i++, j++) {  
        if (A[i] != B[j]) {  
            break;  
        }  
    }  
}
```

```

    }

    if (A[i] == B[j]) {
        printf("Strings are Equal\n");
    } else if (A[i] < B[j]) {
        printf("First string is Smaller\n");
    } else {
        printf("First string is Greater\n");
    }

    return 0;
}

```

Output:

```
First string is Smaller
```

9. Understanding the comparison loop

The loop is:

```

for (i = 0, j = 0; A[i] != '\0' && B[j] != '\0'; i++, j++) {
    if (A[i] != B[j]) {
        break;
    }
}

```

This means:

```

Start i at 0 for string A.
Start j at 0 for string B.
Keep comparing while both strings have not ended.
If a mismatch is found, stop immediately.
Otherwise move both indexes forward.

```

The condition:

```
A[i] != '\0' && B[j] != '\0'
```

means:

Continue only `while` both strings still have real characters.

The line:

```
if (A[i] != B[j]) {  
    break;  
}
```

means:

If the current characters are different, stop comparing.

10. Understanding the final decision

After the loop stops, we check:

```
if (A[i] == B[j])
```

This means:

The current characters are the same.

If both are `\0`, then both strings ended together.

So the strings are equal.

If they are not equal, we check:

```
else if (A[i] < B[j])
```

This means:

The first string is smaller by ASCII order.

Otherwise:

The first string is greater.

11. Dry run: Comparing "Painter" and "Painting"

Strings:

```
A = "Painter"  
B = "Painting"
```

Memory:

```
A: P a i n t e r \0  
B: P a i n t i n g \0
```

Trace:

Index	A[i]	B[j]	Result
0	P	P	Same, continue
1	a	a	Same, continue
2	i	i	Same, continue
3	n	n	Same, continue
4	t	t	Same, continue
5	e	i	Different, stop

At index 5:

```
A[5] = 'e'  
B[5] = 'i'
```

ASCII values:

```
'e' = 101  
'i' = 105
```

Since:

```
101 < 105
```

Final result:

```
First string is Smaller
```

12. Dry run: Equal strings

Strings:

```
A = "code"  
B = "code"
```

Trace:

Index	A[i]	B[j]	Result
0	c	c	Same
1	o	o	Same
2	d	d	Same
3	e	e	Same
4	\0	\0	Both ended

Both strings reach `\0` together.

Final result:

```
Strings are Equal
```

13. Dry run: One string is a prefix of another

Example:

```
A = "abc"
B = "abcd"
```

Trace:

Index	A[i]	B[j]	Result
0	a	a	Same
1	b	b	Same
2	c	c	Same
3	\0	d	Different

At index 3:

```
A[3] = '\0'
B[3] = 'd'
```

The strings are not equal because only A ended.

Since \0 has a smaller value than normal letters, A is considered smaller.

Final result:

```
"abc" is smaller than "abcd"
```

14. Case sensitivity in comparison

String comparison is case-sensitive.

This means uppercase and lowercase letters are different.

Example:

```
A = "Apple"
B = "apple"
```

First characters:

```
A[0] = 'A'  
B[0] = 'a'
```

ASCII values:

```
'A' = 65  
'a' = 97
```

Since:

```
65 < 97
```

Result:

```
"Apple" is smaller than "apple"
```

Important:

This is ASCII order, not normal human dictionary order.

15. Case-insensitive comparison

Sometimes we want:

```
"Apple"
```

and

```
"apple"
```

to be treated as equal.

To do that, we should convert both strings to the same case before comparing.

Example:

```
Apple -> apple  
apple -> apple
```

Then compare:

```
apple == apple
```

So for case-insensitive comparison:

```
Step 1: Convert both strings to lowercase or uppercase.  
Step 2: Compare the converted strings.
```

Part B: Palindromes

16. What is a palindrome?

A palindrome is a string that reads the same forward and backward.

Examples:

```
madam  
level  
liril
```

For `madam`:

Forward:

```
m a d a m
```

Backward:

```
m a d a m
```

They are the same.

So:

```
madam is a palindrome
```

Non-example:

```
hello
```

Backward:

```
olleh
```

Since:

```
hello != olleh
```

`hello` is not a palindrome.

17. Important point about exact matching

By default, palindrome checking is case-sensitive.

Example:

```
Nitin
```

If we compare characters exactly:

```
'N' != 'n'
```

So exact C-style comparison would say it is not a palindrome unless we first normalize the case.

To treat it as a palindrome, convert it to one case first:

```
Nitin -> nitin
```

Then check:

```
n == n  
i == i
```

So it becomes a palindrome after case normalization.

Part C: Palindrome Method 1 — Reverse and Compare

18. Main idea

One simple way to check a palindrome is:

```
Reverse the string.  
Compare the original string with the reversed string.  
If both are the same, it is a palindrome.  
Otherwise, it is not a palindrome.
```

Example 1:

```
Original: madam  
Reverse:  madam
```

Same, so palindrome.

Example 2:

```
Original: hello  
Reverse:  olleh
```

Different, so not palindrome.

19. Reverse-and-compare logic

Steps:

1. Copy the original string backward into another array.
2. Add `'\0'` at the end of the reversed array.

3. Compare the original and reversed strings.
4. If they match, print Palindrome.
5. Otherwise, print Not Palindrome.

This method is easy to understand because it directly uses the meaning of palindrome.

But it uses extra memory because we create another array.

20. Reverse-and-compare complexity

If the string has N characters:

Reversing takes:

$O(N)$

Comparing takes:

$O(N)$

Total time:

$O(N) + O(N) = O(N)$

Extra space:

$O(N)$

because we store the reversed string in another array.

Part D: Palindrome Method 2 — Two-Pointer Method

21. Main idea of the two-pointer method

The two-pointer method checks the string from both ends.

Use two indexes:

```
i = left side  
j = right side
```

For:

```
madam
```

Indexes:

```
Index: 0 1 2 3 4  
Value: m a d a m
```

Start:

```
i = 0  
j = 4
```

Compare:

```
A[0] with A[4]  -> m == m  
A[1] with A[3]  -> a == a
```

Then the indexes meet in the middle.

No mismatch was found, so it is a palindrome.

22. Why the two-pointer method is better

The reverse-and-compare method creates another array.

The two-pointer method does not need another array.

It only uses:

```
int i;  
int j;  
int isPalindrome;
```

So the two-pointer method uses constant extra space.

That means:

```
Extra space = O(1)
```

23. Code: Palindrome using two pointers

```
#include <stdio.h>  
  
int main() {  
    char A[] = "madam";  
    int i, j;  
    int isPalindrome = 1;  
  
    for (j = 0; A[j] != '\0'; j++) {  
        // move j to the null terminator  
    }  
  
    j = j - 1;  
  
    for (i = 0; i < j; i++, j--) {  
        if (A[i] != A[j]) {  
            isPalindrome = 0;  
            break;  
        }  
    }  
  
    if (isPalindrome) {  
        printf("Palindrome\n");  
    } else {  
        printf("Not Palindrome\n");  
    }  
  
    return 0;  
}
```

Output:

Palindrome

24. Understanding `isPalindrome`

The variable:

```
int isPalindrome = 1;
```

means:

Assume the string is a palindrome at first.

If a mismatch is found:

```
isPalindrome = 0;
```

This means:

Now we know the string is not a palindrome.

Then we stop the loop using:

```
break;
```

Why stop early?

Because one mismatch is enough to prove the string is not a palindrome.

25. Understanding how `j` finds the last character

The first loop is:

```
for (j = 0; A[j] != '\0'; j++) {  
}
```

This loop moves `j` until it reaches `\0`.

For:

```
madam
```

Memory:

```
Index: 0 1 2 3 4 5  
Value: m a d a m \0
```

The loop stops when:

```
j = 5  
A[5] = '\0'
```

But index `5` is not a real character.

So we do:

```
j = j - 1;
```

Now:

```
j = 4
```

So `j` points to the last real character:

```
A[4] = 'm'
```

26. Understanding the comparison loop

The comparison loop is:

```
for (i = 0; i < j; i++, j--) {  
    if (A[i] != A[j]) {  
        isPalindrome = 0;  
    }  
}
```

```
        break;
    }
}
```

This means:

```
Start i from the left side.
Start j from the right side.
Compare A[i] and A[j].
If they are different, it is not a palindrome.
If they are same, move i forward and j backward.
Continue until i and j meet or cross.
```

27. Dry run: Palindrome check for "madam"

String:

```
A = "madam"
```

Memory:

```
Index: 0 1 2 3 4 5
Value: m a d a m \0
```

Step 1: Find the end.

```
j stops at index 5, where A[5] = '\0'
```

Step 2: Move back to the last real character.

```
j = j - 1
j = 4
```

Now:

```
i = 0
j = 4
isPalindrome = 1
```

Trace:

Step	i	j	A[i]	A[j]	Result
1	0	4	m	m	Same, continue
2	1	3	a	a	Same, continue

After step 2:

```
i = 2
j = 2
```

The condition:

```
i < j
```

is false because:

```
2 < 2 is false
```

So the loop stops.

No mismatch was found.

Final result:

```
Palindrome
```

28. Dry run: Palindrome check for "hello"

String:

```
A = "hello"
```

Memory:

```
Index: 0 1 2 3 4 5
Value: h e l l o \0
```

After finding the end:

```
j = 4
```

Start:

```
i = 0
j = 4
```

Trace:

Step	i	j	A[i]	A[j]	Result
1	0	4	h	o	Different, stop

Since:

```
h != o
```

we set:

```
isPalindrome = 0
```

Final result:

```
Not Palindrome
```

29. Why the condition is `i < j`

The loop condition is:

```
i < j
```

This means:

```
Keep comparing while the left index is before the right index.
```

When `i` and `j` meet or cross, the checking is complete.

30. Odd-length palindrome example

Example:

```
madam
```

Indexes:

```
0 1 2 3 4
m a d a m
```

Comparisons:

```
A[0] with A[4] -> m == m
A[1] with A[3] -> a == a
```

Middle character:

```
d
```

The middle character does not need to be compared with itself.

So `i < j` stops cleanly when both indexes meet.

31. Even-length palindrome example

Example:

noon

Indexes:

```
0 1 2 3
n o o n
```

Comparisons:

```
A[0] with A[3] -> n == n
A[1] with A[2] -> o == o
```

Then the indexes cross.

Final result:

Palindrome

Part E: Case Normalization Before Palindrome Checking

32. Why case normalization may be needed

By default, this comparison is exact:

```
A[i] != A[j]
```

So uppercase and lowercase letters are different.

Example:

Nitin

First and last characters:

```
'N' and 'n'
```

ASCII values:

```
'N' = 78  
'n' = 110
```

They are not equal.

So exact comparison says:

```
Not Palindrome
```

But if we convert all letters to lowercase first:

```
Nitin -> nitin
```

Then:

```
n == n  
i == i
```

So it becomes palindrome.

33. Beginner case-normalization idea

Before checking palindrome, convert uppercase letters to lowercase.

Pattern:

```
for (i = 0; A[i] != '\0'; i++) {  
    if (A[i] >= 'A' && A[i] <= 'Z') {  
        A[i] = A[i] + 32;  
    }  
}
```

Then run the palindrome check.

This connects Module 2 with Module 6.

Part F: Complexity Analysis

34. String comparison time complexity

String comparison scans characters from left to right.

Worst case:

The strings are equal.

or

The mismatch is at the end.

Example:

```
programming  
programming
```

The loop must check every character.

So time complexity is:

$O(N)$

35. String comparison space complexity

The comparison code uses only a few variables:

```
int i;  
int j;
```

It does not create another array.

So extra space complexity is:

$O(1)$

36. Palindrome two-pointer time complexity

The palindrome method scans to find the end:

$O(N)$

Then it compares characters from both ends.

It checks about half the string:

$N / 2$

In Big-O notation:

$N / 2 = O(N)$

So total time complexity is:

$O(N)$

37. Palindrome two-pointer space complexity

The two-pointer palindrome method uses only a few variables:

```
int i;  
int j;  
int isPalindrome;
```

It does not create another string array.

So extra space complexity is:

$O(1)$

38. Complexity table

Operation	Time Complexity	Extra Space
Compare two strings	$O(N)$	$O(1)$
Palindrome by reverse and compare	$O(N)$	$O(N)$
Palindrome by two pointers	$O(N)$	$O(1)$

Part G: Common Mistakes

39. Mistake 1: Comparing only the first character

Wrong idea:

```
if (A[0] == B[0]) {  
    printf("Strings are Equal");  
}
```

Why this is wrong:

Painter
Painting

Both start with **P**, but they are not equal.

Correct idea:

Compare every character until mismatch or `'\0'`.

40. Mistake 2: Returning equal too early

Wrong idea:

If one pair of characters matches, the strings are equal.

Correct idea:

The strings are equal only **if** all characters match and both reach **'\0'** together.

41. Mistake 3: Forgetting case sensitivity

Example:

```
Apple  
apple
```

These are different in ASCII because:

```
'A' != 'a'
```

Correct idea:

If **case** should not matter, convert both strings to the same case first.

42. Mistake 4: Thinking prefix strings are equal

Example:

```
abc  
abcd
```

The first three characters match:

```
a == a  
b == b  
c == c
```

But then:

First string reaches `'\0'`.
Second string still has `'d'`.

So they are not equal.

43. Mistake 5: Starting `j` at `\0` for palindrome checking

After this loop:

```
for (j = 0; A[j] != '\0'; j++) {  
}
```

`j` points to `\0`.

But `\0` is not a visible character.

So you must do:

```
j = j - 1;
```

Wrong:

Compare `A[i]` with `A[j]` while `j` is still at `'\0'`.

Correct:

Move `j` back to the last real character first.

44. Mistake 6: Not stopping early on palindrome mismatch

Example:

```
hello
```

First comparison:

```
h != o
```

At this point, the string is already not a palindrome.

There is no need to continue.

Correct idea:

```
If one mismatch is found, set isPalindrome = 0 and break.
```

45. Mistake 7: Confusing reverse with palindrome

Every string can be reversed.

But not every string is a palindrome.

Example:

```
hello reversed is olleh
```

This is a valid reverse.

But:

```
hello != olleh
```

So `hello` is not a palindrome.

A palindrome is special because:

```
Original string == Reversed string
```

Part H: Main Code Patterns to Remember

46. Pattern 1: Compare two strings

```
for (i = 0, j = 0; A[i] != '\0' && B[j] != '\0'; i++, j++) {
    if (A[i] != B[j]) {
        break;
    }
}

if (A[i] == B[j]) {
    printf("Strings are Equal");
} else if (A[i] < B[j]) {
    printf("First string is Smaller");
} else {
    printf("First string is Greater");
}
```

47. Pattern 2: Find the last real character of a string

```
for (j = 0; A[j] != '\0'; j++) {
}

j = j - 1;
```

After this:

```
j points to the last visible character.
```

48. Pattern 3: Check palindrome using two pointers

```
int isPalindrome = 1;

for (j = 0; A[j] != '\0'; j++) {
}

j = j - 1;
```

```
for (i = 0; i < j; i++, j--) {  
    if (A[i] != A[j]) {  
        isPalindrome = 0;  
        break;  
    }  
}
```

49. Pattern 4: Print palindrome result

```
if (isPalindrome) {  
    printf("Palindrome");  
} else {  
    printf("Not Palindrome");  
}
```

50. Pattern 5: Normalize case before checking

```
for (i = 0; A[i] != '\0'; i++) {  
    if (A[i] >= 'A' && A[i] <= 'Z') {  
        A[i] = A[i] + 32;  
    }  
}
```

This converts uppercase letters to lowercase.

Then you can do a case-insensitive palindrome check.

Part I: Full Combined Programs

51. Full program: Compare two strings

```
#include <stdio.h>  
  
int main() {  
    char A[] = "Painter";  
    char B[] = "Painting";  
    int i, j;
```

```

for (i = 0, j = 0; A[i] != '\0' && B[j] != '\0'; i++, j++) {
    if (A[i] != B[j]) {
        break;
    }
}

if (A[i] == B[j]) {
    printf("Strings are Equal\n");
} else if (A[i] < B[j]) {
    printf("First string is Smaller\n");
} else {
    printf("First string is Greater\n");
}

return 0;
}

```

Output:

```
First string is Smaller
```

52. Full program: Check palindrome using two pointers

```

#include <stdio.h>

int main() {
    char A[] = "madam";
    int i, j;
    int isPalindrome = 1;

    for (j = 0; A[j] != '\0'; j++) {
    }

    j = j - 1;

    for (i = 0; i < j; i++, j--) {
        if (A[i] != A[j]) {
            isPalindrome = 0;
            break;
        }
    }

    if (isPalindrome) {

```

```

        printf("Palindrome\n");
    } else {
        printf("Not Palindrome\n");
    }

    return 0;
}

```

Output:

```

Palindrome

```

53. Full program: Case-insensitive palindrome check

```

#include <stdio.h>

int main() {
    char A[] = "Nitin";
    int i, j;
    int isPalindrome = 1;

    // Convert uppercase letters to lowercase
    for (i = 0; A[i] != '\0'; i++) {
        if (A[i] >= 'A' && A[i] <= 'Z') {
            A[i] = A[i] + 32;
        }
    }

    // Find the last real character
    for (j = 0; A[j] != '\0'; j++) {}

    j = j - 1;

    // Check palindrome
    for (i = 0; i < j; i++, j--) {
        if (A[i] != A[j]) {
            isPalindrome = 0;
            break;
        }
    }

    if (isPalindrome) {

```

```
        printf("Palindrome\n");
    } else {
        printf("Not Palindrome\n");
    }

    return 0;
}
```

Output:

Palindrome

Part J: Practice Questions

54. Practice 1: Compare two strings

Compare:

Painter
Painting

Expected result:

First string is Smaller

Reason:

The first mismatch is 'e' and 'i'.
'e' has a smaller ASCII value than 'i'.

55. Practice 2: Compare equal strings

Compare:

```
code  
code
```

Expected result:

```
Strings are Equal
```

Reason:

```
All characters match and both strings reach '\0' together.
```

56. Practice 3: Compare prefix strings

Compare:

```
abc  
abcd
```

Expected result:

```
First string is Smaller
```

Reason:

```
The first string ends earlier.
```

57. Practice 4: Explain case sensitivity

Compare:

```
Apple  
apple
```

Question:

Are they equal?

Answer:

No.

Reason:

'A' and 'a' have different ASCII values.

58. Practice 5: Check palindrome

Check:

madam

Expected result:

Palindrome

Reason:

```
m == m  
a == a
```

No mismatch is found.

59. Practice 6: Check non-palindrome

Check:

hello

Expected result:

Not Palindrome

Reason:

h != o

60. Practice 7: Check case-sensitive palindrome

Check exactly:

Nitin

Expected result without case conversion:

Not Palindrome

Reason:

'N' != 'n'

Expected result after converting to lowercase:

Palindrome

Reason:

Nitin -> nitin

61. Practice 8: Trace indexes

For:

level

Fill the trace:

Step	i	j	A[i]	A[j]	Match?
1	0	4	l	l	Yes
2	1	3	e	e	Yes

Final answer:

Palindrome

Part K: Final Summary

62. Key points to remember

Module 6 is about comparing strings and checking palindromes.

String comparison means comparing characters one by one.

Two strings are equal only if all matching characters are the same and both reach `\0` together.

If a mismatch happens, the first different character decides which string is smaller or greater.

ASCII values decide smaller or greater.

String comparison is case-sensitive.

`'A'` and `'a'` are different characters.

A palindrome reads the same forward and backward.

A palindrome can be checked by reversing and comparing.

A better palindrome method uses two pointers.

The two-pointer method compares the first character with the last character, then moves inward.

If any mismatch is found, the string is not a palindrome.

If no mismatch is found before the indexes meet or cross, the string is a palindrome.

String comparison takes $O(N)$ time and $O(1)$ extra space.

Two-pointer palindrome checking takes $O(N)$ time and $O(1)$ extra space.

63. Final mental model

For string comparison:

```
Start at index 0 in both strings.  
Compare characters one by one.  
If they match, move forward.  
If they differ, use ASCII values to decide smaller or greater.  
If both reach '\0' together, the strings are equal.
```

For palindrome checking:

```
Put i at the first character.  
Put j at the last real character.  
Compare A[i] and A[j].  
Move i forward and j backward.  
If any pair is different, it is not a palindrome.  
If the indexes meet or cross with no mismatch, it is a palindrome.
```

64. Module 6 checkpoint

You are ready for Module 7 if you can do these without help:

1. Explain why strings are compared character by character.
2. Write code to compare two strings without using library functions.
3. Explain when two strings are equal.
4. Explain why "abc" and "abcd" are not equal.
5. Compare "Painter" and "Painting" using ASCII order.
6. Explain why "Apple" and "apple" are different.
7. Define what a palindrome is.
8. Check "madam" using two pointers.
9. Check "hello" and stop at the first mismatch.
10. Explain why $j = j - 1$ is needed before palindrome comparison.
11. Explain why the loop condition is $i < j$.
12. State string comparison complexity as $O(N)$ time and $O(1)$ space.
13. State two-pointer palindrome complexity as $O(N)$ time and $O(1)$ space.

Final takeaway:

Module 6 teaches how to compare two strings from left to right and how to check a palindrome by comparing one string from both ends.